Peer-to-Peer Networking and Applications (2021) 14:1071–1087
https://doi.org/10.1007/s12083-020-01062-6

Non-cryptography countermeasure against controllable event triggering attack in WSNs

Haoran Hu[1,2] · **Wei Chang**[1]

Received: 25 August 2020 / Accepted: 26 December 2020 / Published online: 5 February 2021
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC part of Springer Nature 2021

Abstract

Compressive sensing-based data collecting is a promising technique, which can significantly reduce the communication costs in wireless sensor networks (WSNs). However, a recent study shows that it is vulnerable to controllable event triggering attack (CETA): after compromising a sensor and carefully manipulating the environmental elements around a target in the subtree rooted at the sensor, an attacker is able to infer sensitive parameters of the target. The existing countermeasure is purely based on cryptography and requires central control. In this paper, we propose a lightweight non-cryptography-based approach by termly modifying the structure of the data gathering tree. However, how to efficiently and effectively construct the tree is an open problem. To solve the problem, we create a novel topology-based coding scheme and a set of distributed algorithms to mitigate the CETA attack. By adopting this approach, the time for successfully launching a CETA attack is significantly increased. Extensive simulations and experiments show that our solution can efficiently and securely build different data gathering trees for consecutive sensing tasks.

Keywords Compressive sensing · Event triggering attack · Topology representation · Wireless sensor network

1 Introduction

As smart cities are being widely built in many urban areas [1, 2], which use sensors to collect data to optimize infrastructure management, the security of smart cities has been brought into concern [3]. Data gathering, where data collection is accomplished through a tree-based multi-hop routing from sensors towards a common sink node, is a fundamental operation in WSNs. Directly routing individual reading from each sensor to the sink is the most straightforward approach, but it results in a bottleneck of the network lifetime through unbalanced energy consumption: the sensors closer to the sink will run out of energy sooner.

Compressive data gathering (CDG) [4] is a data-gathering approach based on the compressive sampling theory, which reduces the communication costs by exploring the correlations among the sensing data so that the network lifetime is largely prolonged. Instead of sending individual data, each node sends out m weighted sums of the readings of all sensor nodes in its subtree. Eventually, the sink node obtains the weighted sums of all sensor readings, and then accurately reconstructs the original data.

However, CDG is subject to controllable event triggering attack (CETA) [5]. Before using CDG, every sensor node needs to pre-install a random coefficient vector ϕ_j, the values of which are sensitive. Suppose v_j is the target node, which is a member of the subtree rooted at a compromised sensor node v_i. By triggering a controllable event around v_j, such as increasing the surrounding temperature by Δx_i degree, the weighted sum returning through v_i increases by Δy_i accordingly. In this way, the attacker can infer ϕ_j by $\phi_j \approx \Delta y_i / \Delta x_j$.

Paper [5] provides a hash key chain-based countermeasure. In this solution, the sink acts as a central controller, and all sensors must update their coefficient vectors simultaneously. This means the key-based solution requires central control, and inevitably there are key management issues.

In this paper, we propose an approach to defer CETA by dynamically modifying the structure of the data gathering tree. The main idea is that the attacker cannot infer ϕ_j if he is unable to correctly obtain the value of Δy_i corresponding

✉ Haoran Hu
hhrh1h2r3@gmail.com

Wei Chang
wchang@sju.edu

1 Department of Computer Science, Saint Joseph's University, Philadelphia, PA, 19131, USA

2 College of Information Science and Engineering, Hunan Normal University, Changsha, 410081, China

to a controllable event Δx_i. To achieve it, our approach changes the member of every subtree in the whole data gathering tree in consecutive sensing rounds, such that the variance of sensor readings caused by a controllable event is hidden by the variance of sensor readings caused by the modification to the subtree's member.

In some existing works, such as [6, 7], which adopts opportunistic routing, where the next hop (the parent) of a sensor node is selected dynamically from a group of candidates, only those neighbors of v_i which are closer to the sink than v_i can be the candidates of the next hop, thus keeping a loop from forming, which is one of the challenges in our work. Those works are similar to our work in the way that the parents of nodes are also changed during routing. However, those works cannot help mitigate CETA attack because (1) opportunistic routing is in the domain of dynamic routing, where the parents of all nodes are selected dynamically all the time while CETA attack and our work exist in static WSN. (2) The random network topology changes are made by opportunistic routing for the sake of saving energy and bandwidth under the volatile communication conditions of the sensing environment and cannot meet our foremost goal, security, under the threat of CETA attack, which requires the set of members of every subtree gets changed every sensing round. We make every sensor nodes aware of their network topology positions and follow a set of algorithms to update their parents so that they can bring in the network topology changes we need distributedly.

Due to the lack of global information in WSNs, how to securely coordinate the modifications to the return paths among different sensors is an open problem. We face three challenges and solve them in this study:

- How to present and store the topological position information efficiently.
- How to avoid redundant modifications to the data gathering tree under the premise of ensuring that the members of the subtree rooted at every node are different from that in the last round.
- How to avoid forming a loop.

To solve these challenges, we propose a novel form of topological information representation and a set of new algorithms called Subtree Modification Algorithm (SMA), Leaf Parent Changing Algorithm (LPCA), and Partial Scheduling Algorithm (PSA), which are self-organized, distributed and non-cryptography-based.

The remainder of the paper is organized as follows: Section 2 introduces the background knowledge about CDG in WSNs and CETA. Section 3 illustrates our system model on which our solution is based. Section 4 details the implementation of SMA and the solutions to the three aforementioned challenges. Section 5 V introduces a new algorithm LPCA and provides comprehensive evaluation results. Section 6 propose a improved algorithm PSA. Section 7 analyzes the complexity of PSA. Section 8 reviews some related work. Section 9 concludes the paper.

2 Preliminary work

In a WSN, each sensor node acts as both a data originator as well as a data transceiver. Figure 1 shows a data gathering tree using traditional data gathering. Every node forwards its reading and all readings from the sensors in its subtree individually. Note that it's hard to replenish the energy of a sensor after deployment. Once a sensor runs out of its energy, it doesn't function anymore. The closer the node is to the sink, the more times of forwarding it needs to carry out. So in Fig. 1, v1 and v2 are the bottlenecks of the network lifetime.

Note that the environment sensed by a sensor has correlations with those sensed by other sensors. So the sensor readings are correlated with each other. The correlations among readings can be interpreted as a kind of redundancy. Compressive data gathering [4] takes advantage of this nature to compress sensor readings. Figure 2 shows a data gathering tree using compressive data gathering. Compared with traditional data gathering, we can see that besides the reduced communication cost, compressive data gathering has an obvious advantage: load balancing. In compressive data gathering, each sensing round can be divided into m forwarding rounds because it takes each sensor m rounds to forward a single reading to the sink using different coefficients. Denote the coefficient vector pre-installed in v_i as $(\phi_{i1}, \phi_{i2}, ..., \phi_{im})$. Denote the reading of v_i as r_i. Denote the weighted sum calculated by v_i in the kth forwarding round as s_{ik}. Denote the coefficient used by v_i in the kth forwarding round as ϕ_{ik}. Take the route $v_7 \rightarrow v_3 \rightarrow v_1 \rightarrow$

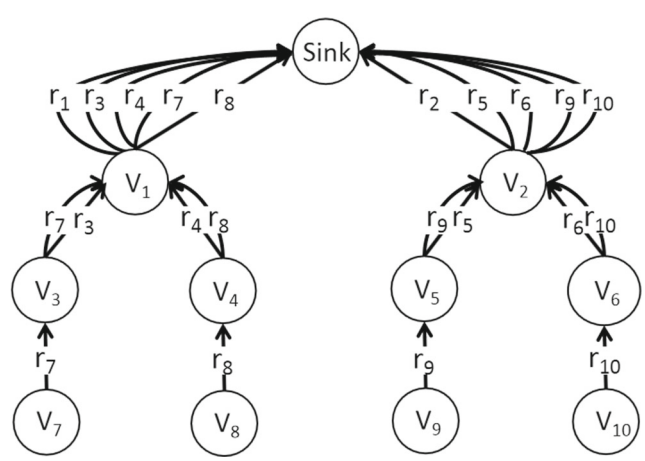


Fig. 1 Traditional Data Gathering

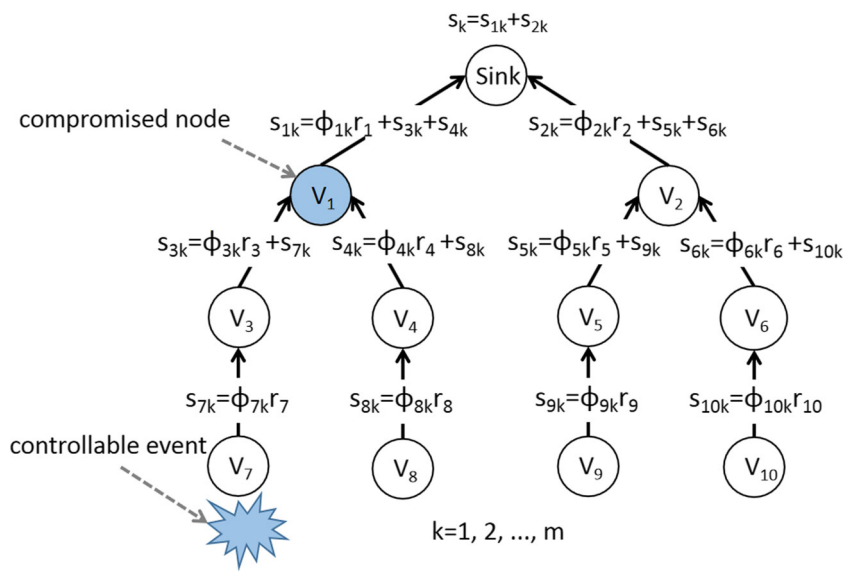


Fig. 2 Compressive Data Gathering

$Sink$ as an example. v_7 is a leaf node. v_7 multiplies its reading r_7 by ϕ_{7k} and then gains s_{7k}. v_7 forwards s_{7k} to v_3. v_3 multiplies r_3 by ϕ_{3k} and pluses the received s_{7k} to gain s_{3k}. v_3 forwards s_{3k} to v_1. v_1 sums up s_{3k} and s_{4k} received from v_3 and v_4 respectively and pluses its weighted reading $\phi_{1k}r_1$ to gain s_{1k}. Eventually, the sink adds s_{1k} and s_{2k} up and then gains the weighted sum of all sensor readings for the kth forwarding round.

After all sensors repeated the process m times using different sets of coefficients, the sink receives m weighted sums of all sensor readings for the current sensing round. The sink has a copy of all coefficients of all sensors. The goal of the sink is to recover n readings from m weighted sums, $\sum_{i=1}^{n} \phi_{ik} r_i$ $(k = 1, 2, ..., m)$, where $m << n$. Moreover, the sink has the pre-learned correlation information, so the sink can recover the original reading of every sensor.

However, CDG is vulnerable to CETA [5]. Note that in different sensing rounds but the same forwarding round, a node uses the same coefficient. In Fig. 2, suppose v_1 is compromised, the attacker can obtain the weighted sums of r_3, r_4, r_7 and r_8, which are propagated through v_1. Suppose the attacker takes v_7 as the target. He triggers a controllable event, like a fire if v_7 is a temperature sensor, around v_7. Then r_7 increases by Δr_7. The s_{1k} increases by Δs_{1k}. Suppose there is no significant natural event happening around v_1, v_3, v_4 and v_8, that is, $\Delta r_i \approx 0, i \in \{1, 3, 4, 8\}$. So Δs_{1k} is mainly caused by Δr_7, that is, $\Delta s_{1k} \approx \phi_{7k} \Delta r_7$. The attacker infers ϕ_{7k} by $\phi_{7k} \approx \Delta s_{1k} / \Delta r_7$ $(k = 1, 2, ..., m)$. Later, by launching CETA targeting at v_3, v_4 and v_8 one by one, the attacker infers the coefficient vectors of all sensors in the subtree rooted at v_1.

Even though sometimes sensor reading r_i can vary significantly without any controllable event being triggered by the attacker, the attacker can reduce its impact on the precision of his calculated coefficient by increasing the number of observations and using some statistical methods.

3 System model

The notations used in the whole paper are listed as follows:

- $TID(v_i)$: The Topology ID of v_i.
- $FID(v_i)$: The Fixed ID of v_i.
- l: A parameter, the length of TID
- k: A parameter. TID can be divided into l/k segments. The length of each segment is k.
- $P(v_i)$: The parent of v_i.
- $S(v_i)$: The set of all nodes in the subtree rooted at v_i. For example, in Fig. 3, $S(v_1) = \{v_3, v_4, v_5, v_7, v_8\}$.
- $C(v_i, j)$: The jth child of v_i.
- $L(v_i, j)$: The set of all nodes at the jth layer in the subtree rooted at v_i. For example, in Fig. 3, $L(v_1, 1)$ is $\{v_3, v_4, v_5\}$.

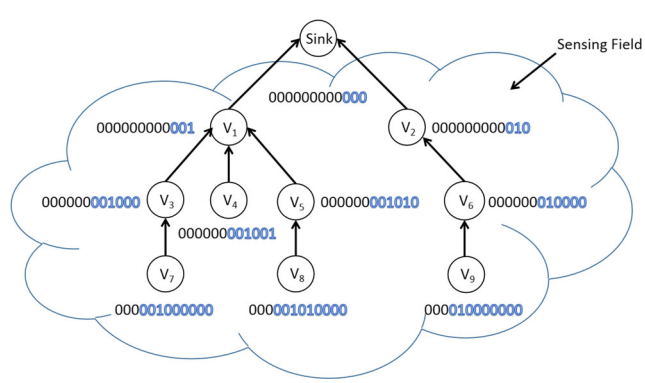


Fig. 3 System Model

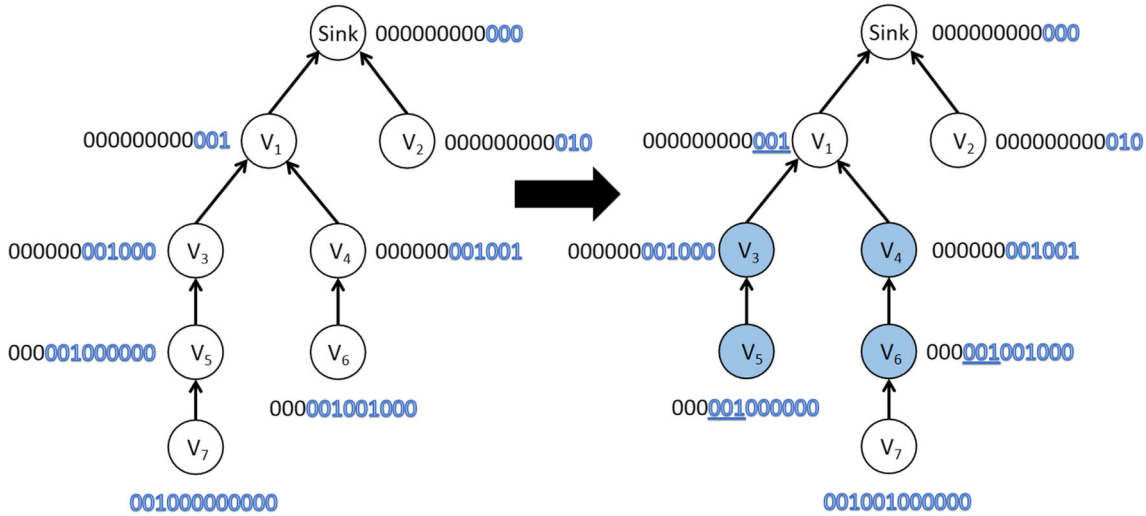


Fig. 4 Update F_{fin}

- $F_{nc}(v_i)$: A flag representing if $S(v_i)$ needs to be changed in the current sensing round.
- $F_{fin}(v_i)$: A flag representing if $S(v_i)$ has been changed in the current sensing round.
- $N(v_i)$: The set of all neighbors of v_i.
- $A(v_i)$: The set of all ancestors of v_i.

Figure 3 shows a data gathering tree in a WSN. In our system model, the out-degree of every node is not fixed, and the out-degrees of different nodes can be different. We only fix the maximum value of every node's out-degree due to our coding scheme which is illustrated in the fourth paragraph of the current section. v_i and v_j are neighbors only if v_i and v_j can communicate bidirectionally, that is, $\|(x_{v_i} - x_{v_j}, y_{v_i} - y_{v_j})\|$<=min($v_i$'s signal range, v_j's signal range).

In our scheme, SMA, $S(v_i)$ of every node v_i changes every sensing round. Suppose an attacker has compromised v_1. In the jth forwarding round of the ith sensing round, the weighted sum gained by the attacker is $\phi_{3j}r_3 + \phi_{4j}r_4 + \phi_{5j}r_5 + \phi_{7j}r_7 + \phi_{8j}r_8$. However, in the $(i+1)^{th}$ sensing round, $S(v_1)$ changes by running SMA. Suppose $S(v_1)$ changes to $\{v_3, v_4, v_5, v_7\}$. In the jth forwarding round of the $(i+1)^{th}$ sensing round, the weighted sum gained by the attacker is $\phi_{3j}r_3 + \phi_{4j}r_4 + \phi_{5j}r_5 + \phi_{7j}r_7$.

In this paper, we design a topology-based coding scheme. Every node has a pair of Fixed ID and Topology ID. The Fixed ID is fixed. TID is presented in the form of a binary string. $TID(v_i)$ contains the topological position information of v_i in bits at the right side and has its left side padded with 0. The length of the right side of $TID(v_i)$ is equal to the product of the hops from v_i to the sink and k. The 01 strings in Figs. 3, 4, 5, 6 are TIDs corresponding to the sensor nodes next to them. When the topology of the data gathering tree changes, the topological position of some nodes changes as well, so their TIDs are updated.

The value of l is assigned according to the maximum number of the sensor nodes that a network can accommodate. Suppose there can be at most N nodes in the network. To make full use of TID, we have $2^{l-1} < N \leq 2^l \implies \log_2 N \leq l < \log_2 N + 1$. So we assign the integer $\in [\log_2 N, \log_2 N + 1)$ to l. Our coding scheme requires that:

- Every non-sink node can have at most 2^k children while the sink can have at most $2^k - 1$ children.

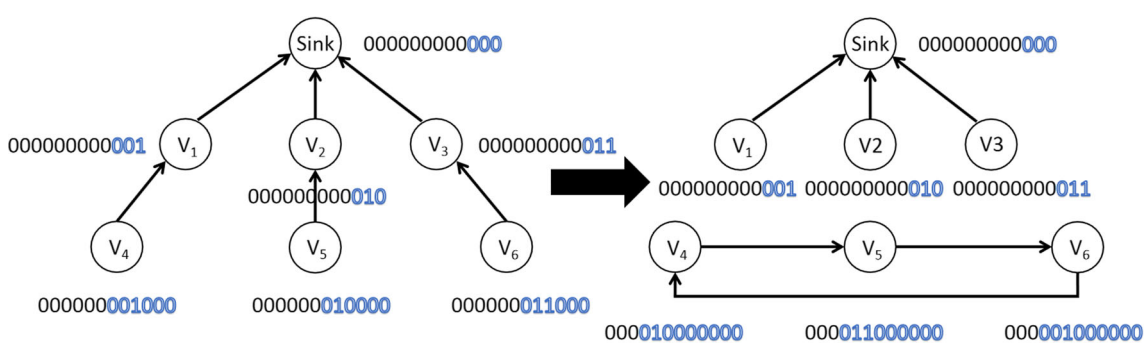


Fig. 5 Loop

- The maximum depth of the data gathering tree is l/k.

The reason why the sink can have at most $2^k - 1$ children instead of 2^k children will be addressed in the second paragraph of Section 4 Subsection 4.2. The value of k is assigned according to the maximum number of the children that every non-sink node can have and the maximum depth of the data gathering tree. For all examples in the remaining paper, $l = 12$, $k = 3$, so there can be at most 4096 sensor nodes in the network, the depth of the data gathering tree can be at most 4, and each node can have at most 8 children. For simplicity and convenience, in the remaining part of this paper, when it comes to TID, we refer to the TID without the sequence of padded leading zeros.

4 Data gathering tree reconstruction

4.1 Subtree modification algorithm

During each sensing round, every sensor runs SMA before the sensor readings are gathered. Algorithm 1 illustrates the general process of SMA.

Algorithm 1 lines 1-8 initialize the procedure. Every non-sink node v_i has a $F_{nc}(v_i)$ flag and a $F_{fin}(v_i)$ flag. At the beginning of the sensing round, every node v_i sets its $F_{fin}(v_i)$ to false and then checks if it is a non-leaf node. If it is, which means $S(v_i)$ is not empty now so that $S(v_i)$ needs to be changed during the adoption of SMA, v_i sets its $F_{nc}(v_i)$ to true. $F_{nc}(v_i)$ is set to false otherwise. $F_{fin}(v_i)$ is set to true if $S(v_i)$ is changed as SMA runs.

Algorithm 1 Subtree modification algorithm (SMA).

1: **for** $i = 0$ to n **do**
2: $F_{fin}(v_i) \leftarrow false$
3: **if** v_i is not a leaf **then**
4: $F_{nc}(v_i) \leftarrow true$
5: **else**
6: $F_{nc}(v_i) \leftarrow false$
7: **end if**
8: **end for**
9: **for** $i = 0$ to n **do**
10: **if** $F_{nc}(v_i) = true$ && $F_{fin}(v_i) = false$ **then**
11: $depth \leftarrow 1$
12: Ask $L(v_i, depth)$ if there is any node that can move out from $S(v_i)$
13: **while** Receive no response **do**
14: $depth \leftarrow depth + 1$
15: Ask $L(v_i, depth)$ if there is any node that can move out from $S(v_i)$
16: **end while**
17: Choose a responder to move out from $S(v_i)$
18: Update F_{fin}, TID of all relative nodes
19: **end if**
20: **end for**

Algorithm 1 lines 9-10 determine whether to change the tree structure. Suppose now v_1 is determining whether it needs to change $S(v_1)$. v_1 finds that $F_{nc}(v_1) = true$ && $F_{fin}(v_1) = false$. $F_{nc}(v_1) = true$ means $S(v_1)$ needs to be changed while SMA is running. $F_{fin}(v_1) = false$ means since SMA in the current round began, none of the

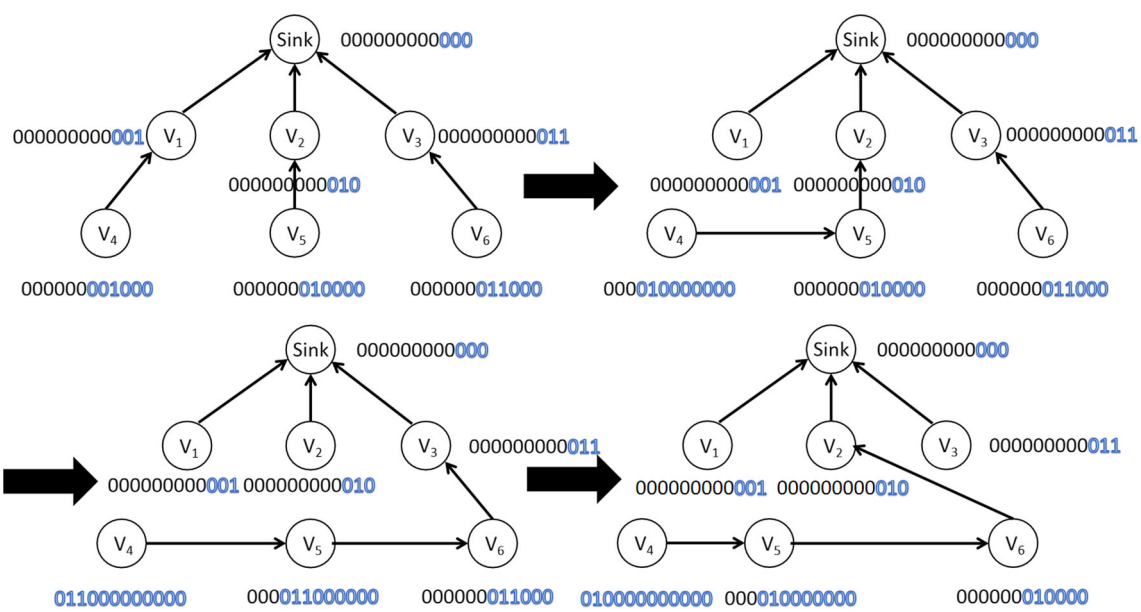


Fig. 6 Loop-free

previous modifications done by other nodes changes $S(v_1)$. So v_1 needs to change $S(v_1)$ by itself now.

Algorithm 1 lines 11-17 modify the subtree structure. v_1 first broadcasts discovery packets to $L(v_1, 1)$ to asks if there is any node v_j that can move out from $S(v_1)$ by changing $P(v_j)$ to a neighbor meeting the following three conditions:

- It is outside $S(v_1)$.
- Its depth in the data gathering tree allows it to have a child since the maximum depth of the data gathering tree is l/k.
- It has less than 8 children, that is, it can accommodate at least one new child.

The nodes in $L(v_1, 1)$ send neighbor query packets to their neighbors one by one. The neighbor receiving the neighbor query packet checks if it meets the above three conditions and replies a neighbor reply packet if qualified or doesn't reply if not qualified. A node will stop querying its remaining neighbors when it receives a neighbor reply packet from the last queried neighbor. Suppose there is no node in $L(v_1, 1)$ that can move out from $S(v_1)$, so v_1 receives no response. Then v_1 asks the remaining layers one by one in the depth-increasing order until it receives at least one response from the last queried layer. Suppose both v_7 and v_8 in $L(v_1, 2)$ respond an offer packet to v_1. Every node v_i can memorize all $S(v_i)$ in the last m sensing rounds and accepts the offer packet sent by the node that can bring it an $S(v_i)$ that is new compared with all $S(v_i)$ in last m sensing rounds, where m is a parameter. If no node can, v_i will accept the one which can bring it an $S(v_i)$ which least recently appeared and send an ACK packet to that node. In this example, v_1 predicts that $S(v_1)$ will become $\{v_3, v_4, v_5, v_8\}$ if it chooses v_7 to move out from $S(v_1)$, and $S(v_1)$ will become $\{v_3, v_4, v_5, v_7\}$ if it chooses v_8 to move out from $S(v_1)$. Suppose v_1 finds that $S(v_1)$ was $\{v_3, v_4, v_5, v_8\}$ in one round of last m sensing rounds while $\{v_3, v_4, v_5, v_7\}$ is new by looking up its memory. So v_1 accepts v_8's offer packet and sends an ACK packet to v_8. v_8 changes $P(v_8)$ from v_5 to v_9. So $S(v_1)$ is changed from $\{v_3, v_4, v_5, v_7, v_8\}$ to $\{v_3, v_4, v_5, v_7\}$.

Algorithm 2 Generation of $TID(C(v_i))$.

```
if Receive TID(v_i) from P(v_i) then
    for j = 0 to C(v_i).length − 1 do
        if v_i is the sink && j = 0 then
            j ← j + 1
        end if
        TID(C(v_i, j)) ← TID(v_i)<<k + j
        Send TID(C(v_i, j)) to C(v_i, j)
    end for
end if
```

Algorithm 3 Find the Length of Right Side of $TID(v_i)$.

```
length ← l − 1
while (TID(v_i)&(1<<length)) = 0 do
    length ← length − 1
end while
length ← length + 1
if length%k ≠ 0 then
    length ← length + k − length%k
end if
```

Algorithm 1 lines 18-20 update F_{fin}, TID of all relative nodes by child update packets, F_{fin} packets, TID update initial packets, and TID update packets.

4.2 Topology ID

When a data gathering tree just initialized, TIDs of all nodes need to be initialized. When TID of a node changed, TIDs of all its descendants need to be updated. Algorithm 2 illustrates the generation of $TID(C(v_i))$. TID of every non-sink ancestor of v_i is a prefix of $TID(v_i)$. $TID(sink)$ is 0, which stays the same all the time. Take the generation of the TID of v_3, v_4 and v_5 in Fig. 3 as an example. v_1 left shifts $TID(v_1)$ 3 bits, and hence 000000001000. Then v_1 adds 000, 001 and 010 to 000000001000 respectively, and hence $TID(v_3)$, $TID(v_4)$ and $TID(v_5)$, which are 000000001000, 000000001001, and 000000001010 respectively. v_1 sends the calculated $TID(v_3)$, $TID(v_4)$ and $TID(v_5)$ to v_3, v_4 and v_5 respectively. Then v_3 and v_5 begin to calculate $TID(v_7)$ and $TID(v_8)$ respectively.

The reasons why the sink can have at most 2^k-1 children instead of 2^k children are (1) 000000000000 is reserved as $TID(sink)$. So none of the sink's children can use 00000000 0000 as its TID. Otherwise, TID will lose its uniqueness. (2) Since the left side of TID is padded with 0, the right side which contains topological position information cannot use 000 as its beginning. Otherwise 000 will be merged into the left side and cannot be distinguished. If a child of the sink uses 000 as the beginning of the right side of its TID, it and all of its descendants will face the aforementioned issue.

As we have mentioned in the third paragraph of Section 3, the right side of TID represents the topological position information while the left side is purely made up of 0. We can extract the topological position information by finding the length of the right side of TID. Algorithm 3 shows the process to find that length.

4.3 Avoiding redundant modifications

In Fig. 4, suppose v_3 informs v_7 of changing $P(v_7)$ to a node that is outside $S(v_3)$. When v_7 changes $P(v_7)$ from v_5 to v_6,

it changes not only $S(v_3)$ but also $S(v_4)$, $S(v_5)$ and $S(v_6)$. So for every node v_i among v_4, v_5 and v_6, if the turn for it to change $S(v_i)$ has not come yet, it doesn't need to change $S(v_i)$ anymore. However, $S(v_1)$ remains unchanged.

Denote the old parent and the new parent of v_i as $P_{old}(v_i)$ and $P_{new}(v_i)$ respectively. We can see that $\{v_3, v_4, v_5, v_6\}$ is $A(P_{old}(v_7)) \cup A(P_{new}(v_7)) - A(P_{old}(v_7)) \cap A(P_{new}(v_7))$, and $\{v_1\}$ is $A(P_{old}(v_7)) \cap A(P_{new}(v_7))$. So we can conclude that:

- As v_i changes $P(v_i)$, $S(v_a)$ changes, where $v_a \in A(P_{old}(v_i)) \cup A(P_{new}(v_i)) - A(P_{old}(v_i)) \cap A(P_{new}(v_i))$,
- As v_i changes $P(v_i)$, $S(v_b)$ remains unchanged, where $v_b \in A(P_{old}(v_i)) \cap A(P_{new}(v_i))$.

The node located at the border of $A(P_{old}(v_i)) \cup A(P_{new}(v_i)) - A(P_{old}(v_i)) \cap A(P_{new}(v_i))$ and $A(P_{old}(v_i)) \cap A(P_{new}(v_i))$ is the crucial node, and we denote it as $Cru(v_i)$. We can use this following feature to find $Cru(v_i)$: $TID(Cru(v_i))$ is the longest common prefix of $TID(P_{old}(v_i))$ and $TID(P_{new}(v_i))$, of which the length is a multiple of k, which is the aforementioned parameter that is assigned 3 in this paper. In Fig. 4, $Cru(v_7)$ is v_1.

In Fig. 4, v_7 sends an inform packet carrying $TID(v_1)$ to v_5 and v_6. v_5 and v_6 forward the packet to v_3 and v_4 respectively. v_3, v_4, v_5 and v_6 update their F_{fin} to true on the receipt of the inform packet. When v_1 receives the packet, it finds that its TID in the packet, which indicates that the members of any subtree rooted at it or any of its ancestors didn't change when v_7 changed $P(v_7)$. So v_1 discards the packet. And the updating process of F_{fin} comes to the end.

Clearly, $Cru(v_i)$ will receive two copies of the inform packet while all other nodes receiving the inform packet will only receive one copy. The reason why we use $TID(Cru(v_i))$ embedded in the inform packet instead of the number of received inform packets to indicate that if a node is $Cru(v_i)$ is that $Cru(v_i)$ won't necessarily receive both two copies of the inform packet at the same time. If it hasn't received the other copy of the inform packet after waiting for a long time, it may assume that it is not the $Cru(v_i)$ and then forward the inform packet to its parent but receive the other copy later. This will cause the F_{fin} of nodes in $A(P_{old}(v_i)) \cap A(P_{new}(v_i))$ to be mistakenly updated to true.

4.4 Loop-free Scheme

When a loop forms in the data gathering tree, nodes in the loop will be cut off from the data gathering tree and keep forwarding readings among themselves instead of forwarding towards the sink. So we design a loop-free scheme to ensure there is no loop forming as the topology of the data gathering tree changes.

Denote the old parent and the new parent of v_i as $P_{old}(v_i)$ and $P_{new}(v_i)$ respectively. Suppose v_j is the ancestor of v_i that lets v_i change $S(v_j)$. To achieve the loop-free scheme, the following two conditions need to be met:

- $P_{new}(v_i) \notin S(v_i)$
- When v_i is changing $P(v_i)$, there cannot be any other nodes changing their parents.

The first condition is necessary since if a node's new parent is its descendant, a loop forms apparently. As we have mentioned as the first condition in the fourth paragraph of Section 4 Subsection 4.1, when v_i changes $P(v_i)$ from $P_{old}(v_i)$ to $P_{new}(v_i)$ to help v_j to change $S(v_j)$, v_i has already made sure that $P_{new}(v_i) \notin S(v_j)$. Also, since v_j is the ancestor of v_i, we have $S(v_i) \subset S(v_j)$. Based on the two conditions, we get $P_{new}(v_i) \notin S(v_i)$. So the first condition is met in SMA. However, the second condition is not met yet.

As the SMA defines, when a node v_i runs SMA, a descendant node in $S(v_i)$ is selected by v_i to change its parent such that $S(v_i)$ gets changed. If we separate the intervals used by sensor nodes to run the SMA, we separate the intervals used by sensor nodes to change their parents, and the second condition is met. If we don't, the intervals used by sensor nodes to change their parents are not exclusive and can be overlapped, that is, multiple nodes changing their parents concurrently is possible.

Before diving into how to separate the intervals used by sensor nodes to run the SMA to meet the second condition, we first introduce a case study with two figures to show that why the second condition is necessary. If we don't impose the second condition as a constraint, there can a special moment when some nodes change their parents simultaneously. Figure 5 illustrates this special case. Suppose every node v_i randomly chooses a moment to run the SMA. Before the modifications to the data gathering tree take place, v_4, v_5 and v_6 are in the same layer. At the same moment, v_1, v_2 and v_3 happen to begin to run the SMA, which makes them change $S(v_1)$, $S(v_2)$ and $S(v_3)$ respectively. As the SMA guides, v_1 selects v_4 to move out from $S(v_1)$, v_2 selects v_5 to move out from $S(v_2)$, and v_3 selects v_6 to move out from $S(v_3)$. v_4 chooses v_5 as $P_{new}(v_4)$ after making sure that $v_5 \in N(v_4)$ and $v_5 \notin S(v_1)$ and $depth(v_5) < 4$ and $C(v_5).length < 8$ as we illustrated as a list of conditions to be met in the fourth paragraph of the Section 4 Subsection 4.1. Similarly, v_5 chooses v_6 as $P_{new}(v_5)$, and v_6 chooses v_4 as $P_{new}(v_6)$. In this case, a loop forms without v_4, v_5 or v_6's notice.

Figure 6 shows how the loop-free scheme is achieved when the second condition is also met. Since we impose the second condition as a constraint, every node v_i selects a node v_j in $S(v_i)$ to change $P(v_j)$ one by one. Note that once

v_i changed $P(v_i)$ and updated $TID(v_i)$, all nodes in $S(v_i)$ will be informed of updating their TIDs. After v_4 chose v_5 as $P_{new}(v_4)$ and v_5 chose v_6 as $P_{new}(v_5)$, TIDs of v_4 and v_5 are updated. When v_6 is choosing a node to be $P_{new}(v_6)$, it finds that $TID(v_3)$ is a prefix of $TID(v_4)$, which indicates that $v_4 \in S(v_3)$ and doesn't meet the first condition listed in the fourth paragraph of the Section 4 Subsection 4.1. So v_6 chooses v_2 instead of v_4 as $P_{new}(v_6)$.

To meet the second condition, we have to specify an order so that every node v_i run the SMA one by one by following this order. We consider two orders. The first order is a pre-fixed order which adopts the time slot. We divide the time which is used to adopt SMA during each sensing round into many time slots. We assign each node a dedicated time slot. During the dedicated time slot of v_i, v_i can decide whether $P(v_i)$ needs to be changed or not, depending on if $F_{nc}(v_i) = true \ \&\& \ F_{fin}(v_i) = false$, and v_i can change $P(v_i)$ if $P(v_i)$ needs to be changed by v_i. Other nodes cannot change their parents at this time slot. The second order is a topology-based order which adopts extended pre-order traversal by keeping to following rules:

- v_i is informed by $P(v_i)$ of changing $S(v_i)$ and informing all of its children of changing theirs.
- v_i won't reply to $P(v_i)$ that it has finished until it changes $S(v_i)$, informs all of its children of changing theirs and receives their replies.
- v_i won't inform the next child until it receives the reply of the last informed child.

Here we compare and contrast these two orders. The advantage of the first order is that the adoption of this order won't bring in additional communication costs. The disadvantage of the first order is that some time slots are partially or completely wasted in some sensing rounds, in other words, the convergence time of SMA required by the first order is more than that by the second order. The reasons are as follows: On the one hand, the duration required by a node v_i to change $S(v_i)$ by itself varies as the topology of the data gathering tree changes, and sometimes the duration of a certain node v_i can even be 0 due to the fact that $S(v_i)$ is empty or has been changed by other nodes. On the other hand, the estimated maximum time slot is assigned to every node since the duration of the time slot has to be determined before the whole compressive data gathering takes place, and we have to make sure the time slot is enough for every node to finish SMA. So v_i's time slot which could have been completely or partially jumped over are wasted in the case that we adopt the first order. Because in the case that we adopt the second order, both the sequence for all nodes to take turns to change the subtree members and the duration for each node to change its subtree members depend on the current topology instead of being pre-fixed. The advantage of the second order is the opposite of the disadvantage of the first order, that is, SMA converges faster by adopting the second order. Because the modifications to the subtree members are scheduled distributedly by the nodes themselves, the disadvantage of the second order is the opposite of the advantage of the first order, that is, when adopting this order, some additional communication costs are introduced.

4.5 Communication cost

For the evaluation of the simulation result, we summarize the packets being used in our solution in this subsection. To get the SMA to run, we have designed 9 kinds of packets. Here is a list of the 9 kinds of packets including their features. The list is based on the assumption that here is v_i's turn to change $S(v_i)$. The case showing how these 9 packets function together in SMA can be found in the fourth and the fifth paragraphs of Section 4 Subsection 4.1.

1) discovery packet
   - direction: from v_i to $C(...C(v_i)...)$ until v_j, $v_j \in L(vi, depth)$
   - content: type, $TID(v_i)$, $FID(C(...C(v_i)...))$, remaining hops to reach the destination
   - length: $4 + 2l + \lceil log_2^{l/k} \rceil$
2) neighbor query packet
   - direction: from v_j to v_t, $v_t \in N(v_j)$
   - content: type, $FID(v_j)$, $FID(v_t)$, $TID(v_i)$
   - length: $4 + 3l$
3) neighbor reply packet
   - direction: from v_t to v_j
   - content: type, $FID(v_t)$, $FID(v_j)$, $TID(v_t)$
   - length: $4 + 3l$
4) offer packet
   - direction: from v_j to $P(...P(v_j)...)$ until v_i
   - content: type, $FID(v_j)$, $FID(P(...P(v_j)...))$, $TID(v_i)$
   - length: $4 + 3l$
5) ACK packet
   - direction: from v_i to $C(...C(v_i)...)$ until v_j
   - content: type, $FID(C(...C(v_i)...))$, $FID(v_j)$
   - length: $4 + 2l$
6) child update packet
   - direction: from v_j to $P_{old}(v_j)$ (or $P_{new}(v_j)$)
   - content: type, $FID(v_j)$, $FID(P_{old}(v_j))$ (or $FID(P_{new}(v_j))$), $TID(Cru(v_j))$

- length: $4 + 3l$

7) F_{fin} update packet

- direction: from $P_{old}(v_j)$ (or $P_{new}(v_j)$) to $P(...P(P_{old}(v_j))...)$ (or $P(...P(P_{new}(v_j))...)$) until $Cru(v_j)$
- content: type, $FID(P(...P(P_{old}(v_j))...))$ (or $FID(P(...P(P_{new}(v_j))...)))$, $TID(Cru(v_j))$
- length: $4 + 2l$

8) TID update initial packet

- direction: from $P_{new}(v_j)$ to v_j
- content: type, $FID(v_j)$, $TID(v_j)$
- length: $4 + 2l$

9) TID update packet

- direction: from v_j to v_k, $v_k \in S(v_j)$
- content: type, $FID(v_k)$, $TID(v_k)$
- length: $4 + 2l$

We evaluate the *cost* as the average number of bytes carried by the fired packets per node. Denote the number of each kind of packets as n_1, n_2, n_3, n_4, n_5, n_6, n_7, n_8, n_9 respectively (The order is the same as that in the list). $cost = (n_1 \times (4 + 2l + \lceil log_2^{l/k} \rceil) + (n_2 + n_3 + n_4 + n_6) \times (4 + 3l) + (n_5 + n_7 + n_8 + n_9) \times (4 + 2l))/8$ (unit: byte).

5 Simulation and experiment

5.1 Baseline approach

Because to the best of our knowledge, there is no existing topology-based countermeasure against CETA, for comparison, we propose a baseline approach to show the significance of our work. In the baseline approach, every non-leaf node chooses the first child node in its children array to change the parent. The selected child will randomly choose a neighbor which is not its current parent to be its new parent. Without topology position information and specific check, after a node changed its parent, a loop may form. Also, even though the node changed its parent without forming a loop, the subtree modification will still be in vain if the new parent is still in the subtree rooted at the old parent due to the fact that the weighted sum gained by the old parent will remain unchanged in this case. Denote the times that nodes change their parents as n_{change}. Denote the times that a loop forms when a node changes its parent as n_{loop}. Denote the times that a node is still in the subtree rooted at the old parent as n_{in}. Denote the times that either a loop forms or a node is still in the subtree rooted at the old parent as n_{fail}. We run a simulation on 20 randomly generated data gathering trees containing 300 nodes, calculate averaged n_{loop}/n_{change}, n_{in}/n_{change}, n_{fail}/n_{change}, and list them as follows:

- $n_{loop}/n_{change} \approx 8\%$
- $n_{in}/n_{change} \approx 7\%$
- $n_{fail}/n_{change} \approx 14\%$

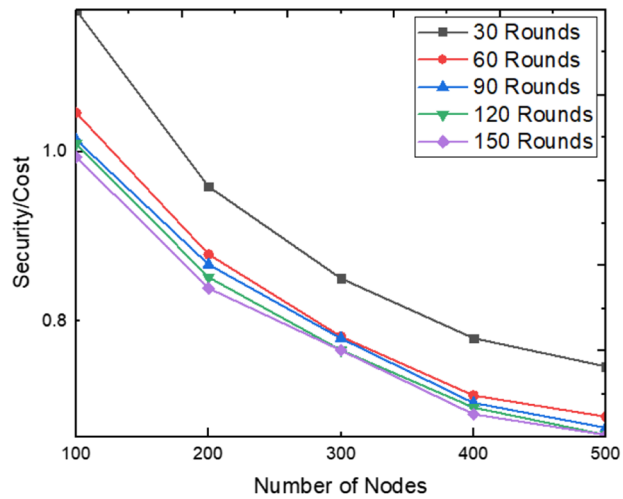
From the data, we can see that the chances of forming a loop or changing parents within the subtree are noteworthy. In our approach, n_{loop}/n_{change}, n_{in}/n_{change}, n_{fail}/n_{change} are all 0 due to proper design. So it is necessary to adopt our approach to change parents with well-designed procedures instead of changing parent arbitrarily.

5.2 Leaf parent changing algorithm

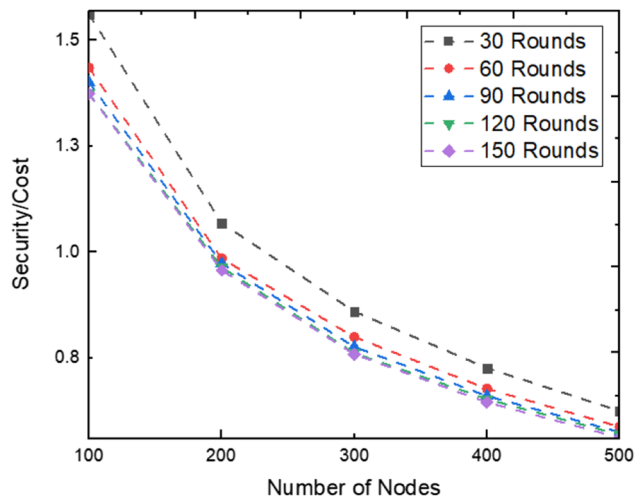
Algorithm 4 Leaf parent changing algorithm (LPCA).

```
1: Run SMA lines 1-8
2: for i = 0 to n do
3:     if v_i is a leaf then
4:         v_i changes its parent
5:         Update TID, F_fin of all relative nodes
6:     end if
7: end for
8: Run SMA lines 9-20
```

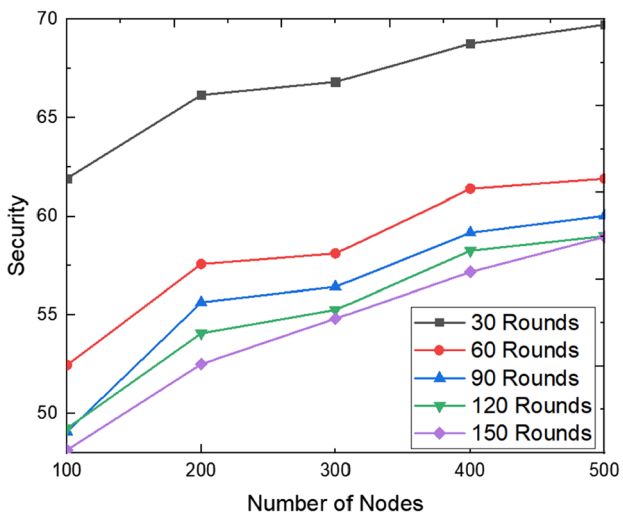
As Section 4 Subsection 4.3 indicates, the deeper the depth of the node v_i which changed $P(v_i)$ is, the larger $A(P_{old}(v_i)) \cup A(P_{new}(v_i)) - A(P_{old}(v_i)) \cap A(P_{new}(v_i))$ may be, the fewer modifications have to be done to the topology of the data gathering tree. So we create another algorithm called Leaf Parent Changing Algorithm (LPCA), which can be divided into two phases. In the first phase (LPCA lines 1-7) of LPCA, when it comes to v_i's time slot, v_i needs to change $P(v_i)$ if and only if it is a leaf node currently. If v_i needs to change $P(v_i)$, v_i will choose a neighbor whose depth in the data gathering tree allows it to have children and which has less than 8 children (that is, can accommodate a new child) to be $P_{new}(v_i)$. In this way, a group of leaf nodes takes initiative to change their parents instead of being scheduled to do so by an ancestor node in SMA. This causes the subtree members of a part of non-leaf nodes to get changed passively in LPCA unlike that in SMA where the subtree members of a part of non-leaf nodes are scheduled to be changed by the non-leaf nodes themselves. However, the first phase of LPCA cannot guarantee that every node v_i changes $S(v_i)$ while SMA can. To achieve that, LPCA has the second phase. In the second phase (LPCA lines 8), we run SMA on those nodes whose $F_{fin} = false$ (that is, of which the subtree members are yet to be changed).



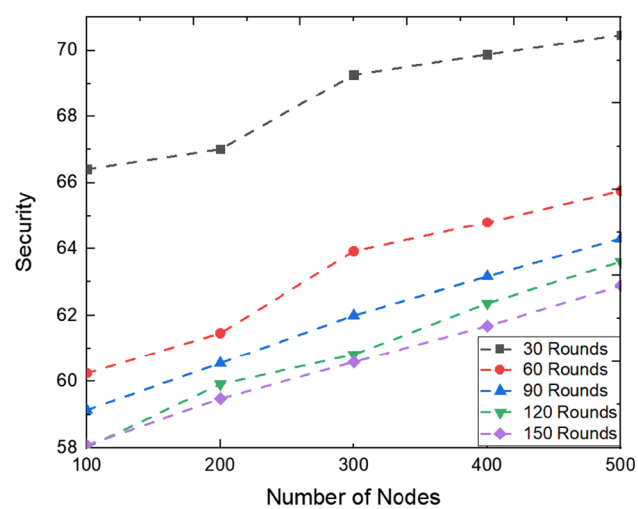
(a) SMA: Security/Cost



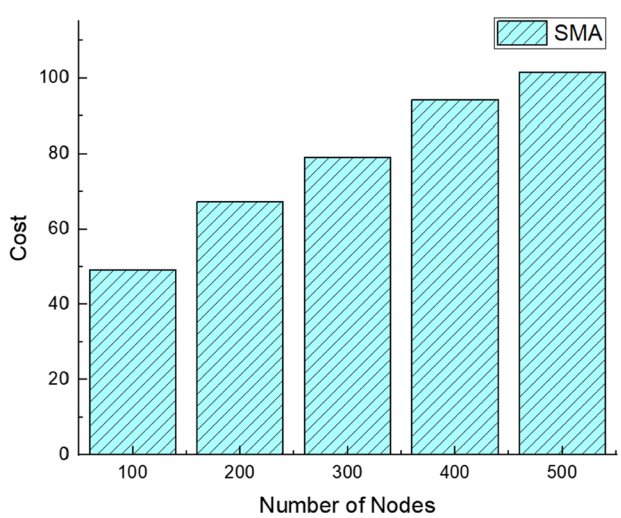
(b) LPCA: Security/Cost



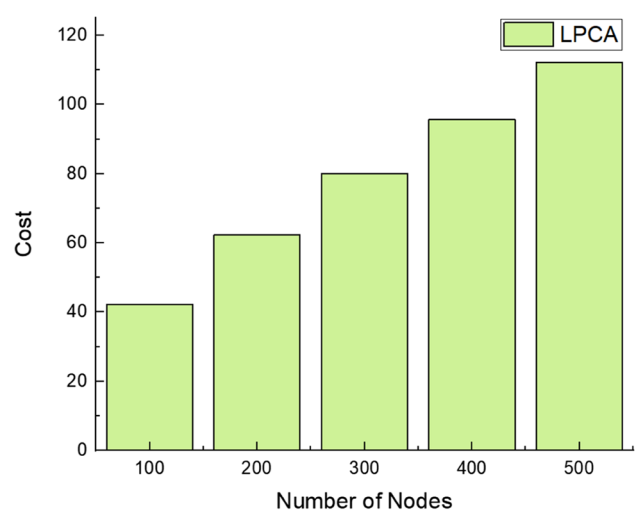
(c) SMA: Security



(d) LPCA: Security



(e) SMA: Cost



(f) LPCA: Cost

Fig. 7 Number of Nodes

5.3 Simulation

5.3.1 Simulation setup

We use struct in C++ to simulate the storage of every sensor node. The contents of the struct that simulate v_i's storage are as follows: (1) v_i's TID, (2) v_i's FID, (3) v_i's coordinates, (4) $P(v_i)$'s TID, (5) FIN flag, (6) NC flag, (7) an array that stores v_i's neighbors' FIDs, (8) an array that stores v_i's children's FIDs, (9) an array that stores the sets of the FIDs of all v_i's previous descendants, which is like a series of snapshots of all v_i's previous subtrees, (10) a flag to record if v_i's subtree is updated to a new subtree, that is, a subtree that cannot be found in the aforementioned array that stores the previous subtrees, in current sensing round. Note that (3)(10) are only used to simulate and evaluate our algorithms and are not needed when the sensor nodes are deployed in real environment for data collection, (9) are partly used while simulating the sensor nodes because we set the senor nodes' memory to be limited to only store the previous subtrees in last m sensing rounds and fully used while simulating the attacker because we assume that the memory used by the attacker is not limited.

We run SMA and LPCA on two copies of the same randomly generated data gathering tree for 150 sensing rounds, where (1) the spanning of the data gathering tree is accomplished through Breath-First Search and (2) the signal range and the coordinate of every node are generated by a random number generator conforming to the uniform distribution. Before running SMA and LPCA, we let every sensor check if its number of neighbors reaches a threshold, which is a parameter called the minimum number of neighbors. When its number of neighbors doesn't reach the threshold, to reduce the risk of being isolated as SMA and LPCA run and the topology of the data gathering tree changes, we enable it to increase its signal range gradually until its number of neighbors reaches the threshold. The observation window is 30 sensing rounds. After each complete observation window, we collect the average data for both $security$ and $cost$ for the latest 30 sensing rounds. So we observe 5 times during 150 sensing rounds. For each chosen value of the parameter to be plotted as the x-axis on the graph, we run SMA and LPCA on 20 different randomly generated data gathering trees and calculate the average value of the results.

Standard parameters: the number of nodes=300, the minimum number of neighbors=10, sensing area=100 × 100 (that is, can accommodate 10000 nodes), the capacity of memory for every node to record previous subtrees=10 rounds, k=3, the maximum number of children=2^k=8, l=30 (l is the length of TID).

As the attacker only needs to monitor the weighted sum from a single subtree, we assume that he has enough memory to record the weighted sums for all sensing rounds. Denote the number of node v_i whose $F_{nc}(v_i) = true$ and whose $S(v_i)$ has been changed to an $S(v_i)$ which is new in all previous sensing rounds as s_1. Denote the number of node v_i whose $F_{nc}(v_i) = true$ as s_2. We evaluate the $security$ as s_1/s_2 (unit:%). As shown in last paragraph, $l = 30, k = 3$, so we have: $cost = (n_1 \times (4+2l+\lceil log_2^{l/k} \rceil) + (n_2 + n_3 + n_4 + n_6) \times (4 + 3l) + (n_5 + n_7 + n_8 + n_9) \times (4 + 2l))/8 = (68n_1 + 94(n_2 + n_3 + n_4 + n_6) + 64(n_5 + n_7 + n_8 + n_9))/8$ (unit: byte). We evaluate the performance of both SMA and LPCA as $security/cost$.

5.3.2 Simulation results analysis

From the horizontal direction of Fig. 7c and d, we can see that as the number of nodes increases, the $security$ of both SMA and LPCA increases. We find that the increment of the $security$ of LPCA is caused by the increasing percentage of nodes that changed their parents, which is shown in Table 1.

Because parent changes mostly happen in the first phase of LPCA according to the simulation results, and nodes that changed their parents in the first phase are leaf nodes, this can also be interpreted as the higher the number of nodes is, the higher the percentage of leaf nodes is. More nodes that changed parents mean more modifications to the topology of the data gathering tree. And this indicates that $S(v_i)$ of every non-leaf node tends to be changed more times in a single round of LPCA. So at the end of LPCA, $S(v_i)$ of every non-leaf node v_i is more likely to appear as an $S(v_i)$ which is new in all previous sensing rounds.

However, according to the simulation result, the percentage of nodes that changed their parents in SMA doesn't show the tendency of increasing along with the increasing number of nodes. Note that SMA is stateful, where every non-leaf node v_i takes initiative to choose a descendant that can bring it a new $S(v_i)$ by moving out from $S(v_i)$. So there must be an additional factor correlated with the $security$ of SMA besides the aforementioned factor that is correlated with the $security$ of both SMA and LPCA. This additional factor is the number of nodes that send the offer packet to

Table 1 Percentage of Nodes Changing Parents in LPCA

Number of Nodes	Percentage of Nodes Parents in LPCA
100	68.7313
200	72.9115
300	74.5321
400	75.8327
500	76.7266

Table 2 Subfactor (2)

Number of Nodes	Subfactor (2)
100	3.16048
200	3.86085
300	4.22676
400	4.52064
500	4.61998

Table 3 Percentage of Each Failure

Type of Failure	Percentage of Each Failure	
	SMA	LPCA
Failure 1	96.9312	99.9688
Failure 2	0	0.000033517
Failure 3	3.0688	0.0311372

the ancestor v_i of which $S(v_i)$ is to be changed. The correlation can be interpreted as that the more nodes which reply an offer packet to the ancestor are, the more options the ancestor has when choosing a node to move out from the subtree, the less the repetitive rate based on the subtrees in all previous sensing rounds can be, the higher the $security$ of SMA is. And this factor is the product of two subfactors:

- subfactor (1): The percentage of nodes that send the offer packets to v_i among those nodes which receive the discovery packets from v_i.
- subfactor (2): The number of nodes that receive the discovery packets from v_i.

According to the simulation result, we have three findings:

- finding (1): The average number of neighbors of every node is about 30% of the total number of nodes.
- finding (2): All nodes receiving the discovery packets reply with an offer packet.
- finding (3): The discovery packet always only needs to be sent to the first layer of the subtree.

Based on finding (1), finding (2) is reasonable since each node tends to have so many neighbors that it can almost always find a neighbor meeting the three conditions to be the new parent. Based on finding (2) and the fact that v_i will only send the discovery packet to the next layer in its subtree only if it received no response from the last queried layer, finding (3) is quite sensible. So based on finding (1), (2) and (3), we know that subfactor (1) is almost always 100% so it doesn't impact the $security$ of SMA. So we can predict that it is subfactor (2) that impacts the $security$ of SMA. Table 2 shows that subfactor (2) does increase with the increasing number of nodes, which verifies our prediction.

From the vertical direction of Fig. 7c and d, we can see that as the duration increases, the $security$ of both SMA and LPCA goes down since the solution space of new subtrees is consumed, and the possibility of each node v_i repeating $S(v_i)$ increases. As a result, the possibility that the attacker infers the coefficients increases as time elapses.

From Fig. 7e and f, we can see that as the number of nodes increases, the $cost$ of both SMA and LPCA increases. Figure 8 shows the percentage of $cost$ that each kind of packets accounts for in SMA and LPCA when the number of nodes is equal to 300 (The percentage is similar when the number of nodes is equal to other values). Note that the packet transmission in the second phase of LPCA accounts for negligible amount of $cost$ so we only plot those in the first phase on Fig. 8. We can see that the main packet contributing to the $cost$ of both SMA and LPCA is the neighbor query packet. To exploring the reason why the $cost$ of both SMA and LPCA increases, we study the factors with which the number of neighbor query packets is positively correlated, which is listed as follows:

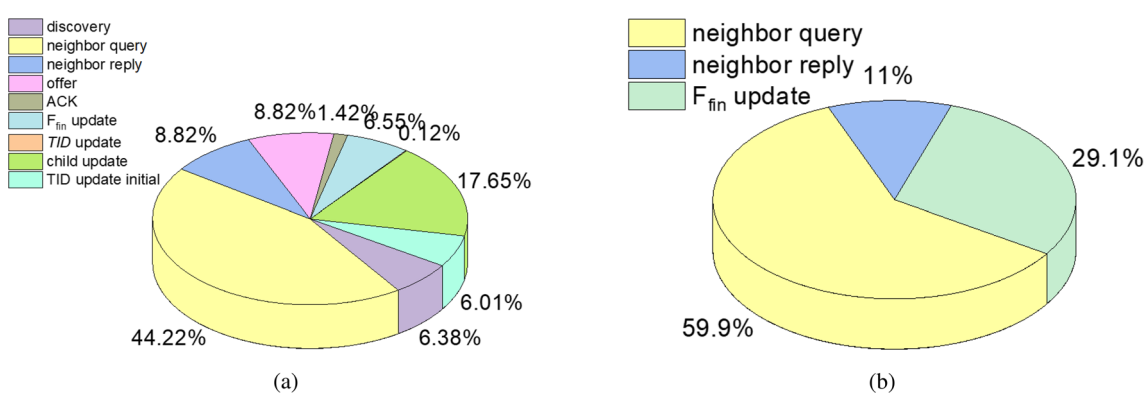


Fig. 8 Cost Components

Table 4 Percentage of Insufficient-room Neighbors

Number of Nodes	Percentage of Insufficient-room Neighbors	
100	8.61652	5.63229
200	10.0677	7.92652
300	10.4018	8.71618
400	10.7597	9.19432
500	10.9801	9.63317

- factor (1): The number of nodes that receive the discovery packets.
- factor (2): The possibility for a queried neighbor that fails to meet the conditions to become the new parent.

As we have mentioned as Table 2, factor (1) increases as the number of nodes increases. In LPCA, factor (1) doesn't exist, and the *cost* of LPCA is only correlated with factor (2). To see factor (2)'s influence on the *cost* of SMA and LPCA, we first discover the failure of which condition among the three conditions accounts for the most of the failures of being the new parent when the number of nodes is equal to 300 (The percentage is similar when the number of nodes is equal to other values). In Table 3, failure 1, 2, 3 stand for not having enough room for new children, the depth reaching the maximum allowed value and the neighbor being in the subtree to be changed respectively. Note that in LPCA, failure 3 only exists in the second phase. In Table 3, we also count the failures happening in the second phase in LPCA.

We can see that when a neighbor fails to meet the conditions, it is mostly because the neighbor doesn't have enough room for new children. So the average percentage of neighbors not having enough room can be an indicator of factor (2). From Table 4, we can see the changing tendency of percentage of insufficient-room neighbors as the number of nodes increases.

Table 4 shows that after the number of nodes reached 200, there is no significant increment of the average percentage of neighbors not having enough room in SMA, which is an indicator of factor (2). So the increment of *cost* in SMA is mainly attributed to the increment of factor (1). But we can see a considerable increasing tendency of factor (2) in LPCA with the increasing number of nodes. So the increment of *cost* in LPCA is mainly attributed to the increment of factor (2).

Figure 8a shows that the TID update packet accounts for few in the *cost* of SMA. Here we illustrate the reason. Denote n_i as the number of nodes whose out-degree is i. A node can have at most 2^k children. So the possible values of out-degree of nodes $\in [0, 2^k]$. We have:

$$n = \sum_{i=0}^{2^k} n_i \tag{1}$$

Also, we know the number of edges is equal to the number of nodes minus 1, that is, $n - 1$, and the number of edges is equal to the sum of out-degrees of all nodes. So we have:

$$n - 1 = \sum_{i=1}^{2^k} i n_i \tag{2}$$

Based on Eqs. 1 and 2, we get:

$$n_0 = 1 + \sum_{i=2}^{2^k} (i-1) n_i \tag{3}$$

A node whose out-degree is 0 is a leaf node. From Eq. 3, we can see that the number of leaf nodes accounts for a large percentage of the total number of nodes. Since there is no

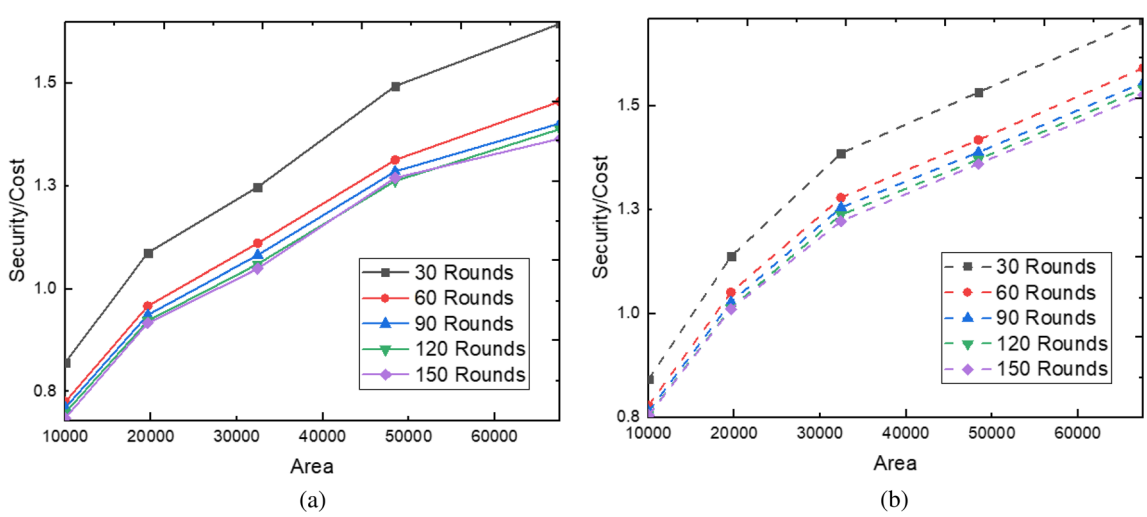


Fig. 9 Area (Simulation)

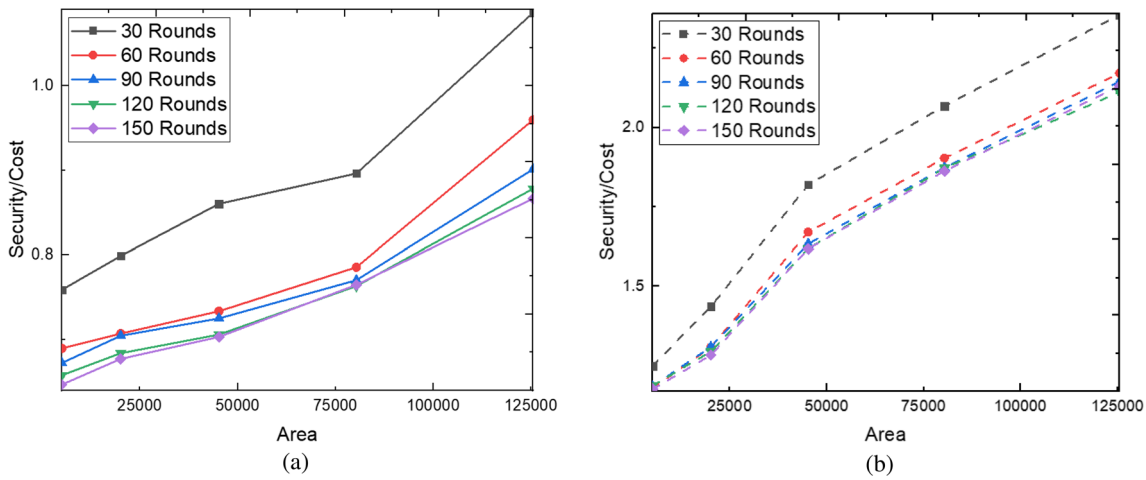


Fig. 10 Area (Experiment)

bias about the out-degree of v_i when the ancestor v_j chooses its descendant v_i to change $P(v_i)$, we can conclude that most of the nodes which change their parents are leaf nodes. Also, we know that when a node v_i changes $P(v_i)$, v_i needs to inform all nodes in $S(v_i)$ of updating TID, and here is when the forwarding of TID update packets happens. Since $S(v_i)$ of a leaf node v_i is empty, it is sensible that there are few usages of TID update packets in SMA.

From Fig. 7a and b, we can see that as the number of nodes increases, the performance of both SMA and LPCA decreases since the $cost$ increases faster than the $security$ does. Figure 9 shows that the larger the sensing area is, the higher the performance of both SMA and LPCA is. So we can conclude that the sparser the nodes are, the better SMA and LPCA perform.

5.4 Experiment

Dataset [8] contains positions of 54 real sensors deployed in a lab. We experiment by running our simulation codes on the data gathering trees generated with the relative sensor positions the same as those in the dataset. By the relative position of a sensor, we mean the position relative to other sensors. We scale the absolute position of every sensor with an increasing proportion, and in the meantime, keep the relative position the same. In other words, we zoom in the topology graph of the data gathering tree gradually. We run SMA and LPCA on 20 data gathering trees with randomly generated signal ranges for each set of absolute sensor positions and calculate the average data with the same criteria of $security$ and $cost$ as those in the simulation. From Fig. 10, we can see that on the real dataset, our conclusion that "the sparser the nodes are, the better SMA and LPCA perform" still holds.

6 Extension and further discussion

Algorithm 5 Partial scheduling algorithm (PSA).

1: Run SMA lines 1-8
2: Randomly generate wn numbers that range over all nodes' FIDs
3: **for** $i = 0$ to n **do**
4: **if** v_i is a leaf && $FID(v_i) \in wn$ randomly generated numbers **then**
5: Change $P(v_i)$
6: Update TID, F_{fin} of all relative nodes
7: **end if**
8: **end for**
9: Run SMA lines 9-20

According to the simulation and experiment results, The performance of SMA and LPCA are close to each other. However, we know that the subtree modification operation in the first phase of LPCA changes nodes' parents without any query and response beforehand like that in SMA does. Also, we know that SMA has a mechanism to avoid redundant modifications while LPCA doesn't, so we suspect that changing all leaf parents in LPCA may not be necessary, which will bring too many redundant modifications. SMA is based on scheduling with a fixed set of rules while LPCA is opportunistic in its first phase. We predict that there is a balance between SMA and LPCA, which is the point that the algorithm achieves the best performance. Based on this intuition, we design a new algorithm called Partial Scheduling Algorithm (PSA). In PSA, we introduce a parameter w, which is a percentage. We embed a seed and a random number generator in every node. All nodes share the same seed and the same random number generator.

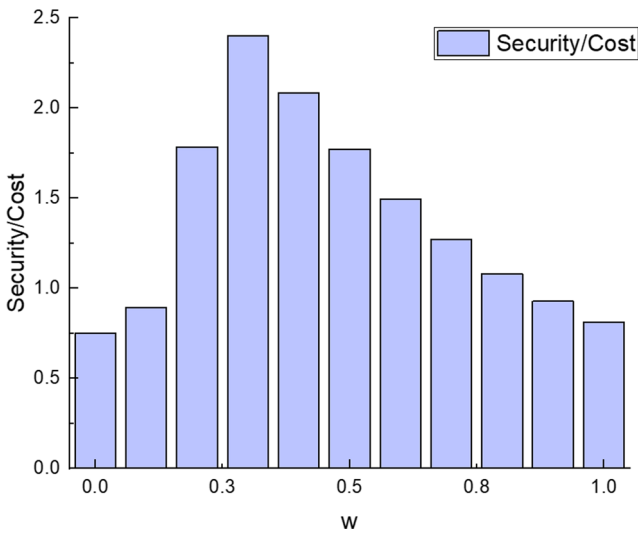


Fig. 11 PSA: Security/Cost

At the beginning of every sensing round, every node uses the seed and the random number generator to generate wn numbers that range over all nodes' FID, where n is the number of nodes. The remaining process is similar to LPCA. The only difference is that in PSA, a node v_i will change $P(v_i)$ only when it is a leaf node and its FID is inside the numbers randomly generated in this sensing round. In this way, since the random number generator has no bias on if the node is a leaf node or not, there will be approximately wn_{leaf} leaf nodes changing parents, and thus avoiding redundant modifications when w is properly set. We run a simulation to verify our prediction, find the optimal w and plot the average results as Fig. 11. PSA is SMA when $w = 0$, and PSA is LPCA when $w = 1$. From Fig. 11, we can see that PSA performs best when $w = 0.3$. The performance of PSA will go down no matter in which trend w changes. When w changes towards 0 from 0.3, PSA converges to SMA and inevitably results in more aforehand queries and responses as the scheduling process in SMA lines 9-20 becomes more and more. When w changes towards 1 from 0.3, PSA converges to LPCA and incurs more and more redundant changes of leaf parents.

7 Analysis of PSA complexity

n is the number of nodes. Algorithms 2 and 3 are invoked as PSA runs. The time complexity of Algorithm 2 is $O(1)$. Due to the fact that we fix the length of TID and FID according to the maximum number of nodes the network needs to accommodate, the time complexity of Algorithm 3 is $O(1)$. We first calculate the time complexity of PSA.

1) PSA Line 1 (SMA Lines 1-8): The time complexity is $O(n)$.
2) PSA Lines 2-8: Every time a leaf node changes its parent, there will be $O(n)$ nodes that get triggered to forward F_{fin} update packets and run Algorithm 3. The number of leaf nodes that change parents is $O(wn)$. So the time complexity of PSA is $O(wn)O(n)O(1) = O(wn^2)$.
3) PSA Line 9 (SMA Lines 9-20):
   - SMA Lines 11-17: According to the simulation and analysis, $depth$ in SMA is always 1. In line 15 of SMA, Algorithm 3 will be invoked. The number of nodes that receive the discovery packets and run Algorithm 3 is less than the maximum out-degree of a node, which is a constant, that is, $O(1)$. So the time complexity of lines 11-17 is $1O(1)O(1) = O(1)$
   - SMA Line 18: In line 18 of SMA, Algorithms 2 and 3 are invoked. According to the simulation and analysis, the number of nodes that run Algorithm 2 to update the TIDs is almost 0. For a node that needs to run Algorithm 2, the times of it running Algorithm 2 is equal to the out-degree of the node, of which the maximum is a constant, that is, $O(1)$. The nodes that run Algorithm 3 are the nodes forwarding F_{fin} update packets, of which the number is $O(n)$. So the time complexity of line 18 is $mO(1)O(1) + O(n)O(1) = O(n)$ $(m \approx 0)$.

   So the time complexity of PSA Line 9 (SMA Lines 9-20) is $O((1-w)n)(O(1) + O(n)) = O((1-w)n^2)$

The total time complexity of PSA is $O(n) + O(wn^2) + O((1-w)n^2) = O(n^2)$. The time complexity of running PSA by each node is $O(n)$.

The contents of every sensor node's storage are specified in the simulation setup, and they are not correlated with n. So the space complexity of running PSA by each node is $O(1)$, and the space complexity of PSA is $O(n)$.

8 Related work

In compressive sensing, the sink recovers the sensor readings x by solving a l_1-norm minimization problem,

$$\min\|d\|_{l1} \quad s.t. \quad y = \Phi x, \quad x = \Psi d \tag{4}$$

where y is the vector of weighted sums, Φ is the measurement matrix, which is composed of rows of coefficient vectors, x is the vector of original readings, Ψ is the sparsifying basis, and d is the vector of readings represented on Ψ. Two prerequisites are needed for compressive sensing:

- sparsity: The signals represented on Ψ should be sparse enough to have the potential to be compressed. A sparsifying basis Ψ can be generated by discrete cosine transform (DCT) or discrete wavelet transform (DWT).

Also, paper [9] proposed a dictionary learning framework to generate a proper sparsifying basis Ψ.

- incoherence: The more incoherence between Φ and Ψ is, the fewer measurements are required to reconstruct the original readings [10].

Compressive sensing has been viewed as a symmetric cryptosystem where the measurement matrix plays as the key, which is first proposed in [11]. So the security of the measurement matrix is critical. Initially, many security models in compressive sensing, such as [12, 13], are based on the strategy that the measurement matrix is updated every sensing round. However, updating the measurement matrix frequently can result in huge consumption of sensor energy and network bandwidth. So alternatively, recent studies that apply encryption to the weighted sum emerged. Paper [14] adopts homomorphic encryption, which allows the data to be aggregated in the cipher domain instead of being decrypted and then being aggregated. Paper [15] combines lattice in group theory with compressive sensing, which can bring ciphertext indistinguishability. Paper [16] introduces information embedding mechanism to achieve privacy preserving.

9 Conclusion

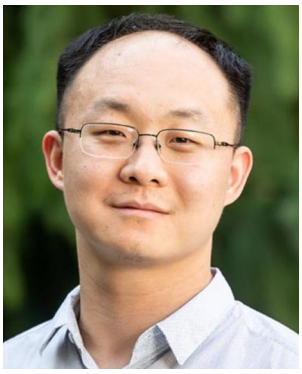
WSNs play an essential role in smart cities. CDG in WSNs can improve communication efficiency and network lifetime. However, CDG is vulnerable to CETA. In this paper, we find that having a fixed set of subtree members of a compromised node is the prerequisite for launching a CETA attack; otherwise, the difference before and after a controllable event will not be able to be fully observed at the compromised node. Therefore, for a series of sensing tasks towards the sink node, the CETA attack is useless if the members of every subtree in the data gathering tree are always different. In this paper, we depict a novel form of topological information representation and illustrate a set of self-organized, distributed and non-cryptography-based algorithms (SMA, LPCA, PSA), which continuously change the members of every subtree in the data gathering tree. Extensive simulation and experiment results show that our approaches can efficiently and effectively increase the security in the compressive sensing-based data gathering process.

References

1. Anthopoulos L (2017) Smart utopia vs smart reality: Learning by experience from 10 smart city cases. Cities 63:128–148
2. Silva BN, Khan M, Han K (2018) Towards sustainable smart cities: a review of trends, architectures, components, and open challenges in smart cities. Sustain Cities Soc 38:697–713
3. Zhang K, Ni J, Yang K, Liang X, Ren J, Shen XS (2017) Security and privacy in smart city applications: Challenges and solutions. IEEE Commun Mag 55(1):122–129
4. Luo C, Wu F, Sun J, Chen CW (2009) Compressive data gathering for large-scale wireless sensor networks. In: Proceedings of the 15th annual international conference on mobile computing and networking, MobiCom '09, pp 145–156, New York, NY, USA. Association for Computing Machinery
5. Hu P, Xing K, Cheng X, Wei H, Zhu H (2014) Information leaks out: attacks and countermeasures on compressive data gathering in wireless sensor networks. In: IEEE INFOCOM 2014 - IEEE conference on computer communications, pp 1258–1266
6. Zeng K, Yang Z, Lou W (2009) Location-aided opportunistic forwarding in multirate and multihop wireless networks. IEEE Trans Veh Technol 58(6):3032–3040
7. Liu X, Zhu Y, Kong L, Liu C, Gu Y, Vasilakos AV, Wu M (2015) CDC: Compressive data collection for wireless sensor networks. IEEE Trans Parallel Distrib Syst 26(8):2188–2197
8. Bodik P, Wei H, Guestrin C, Madden S, Paskin M, Thibaux R (2004) Intel lab dataset. http://db.csail.mit.edu/labdata/labdata.html
9. Duarte-Carvajalino JM, Sapiro G (2009) Learning to sense sparse signals: Simultaneous sensing matrix and sparsifying dictionary optimization. IEEE Trans Image Process 18(7):1395–1408
10. Donoho DL, Huo X (2001) Uncertainty principles and ideal atomic decomposition. IEEE Trans Inf Theory 47(7):2845–2862
11. Candes EJ, Tao T (2006) Near-optimal signal recovery from random projections: Universal encoding strategies? IEEE Trans Inf Theory 52(12):5406–5425
12. Bianchi T, Bioglio V, Magli E (2014) On the security of random linear measurements. In: 2014 IEEE international conference on acoustics, speech and signal processing (ICASSP), pp 3992–3996
13. Cambareri V, Mangia M, Pareschi F, Rovatti R, Setti G (2015) Low-complexity multiclass encryption by compressed sensing. IEEE Trans Signal Process 63(9):2183–2195
14. Zhang P, Wang S, Guo K, Wang J (2018) A secure data collection scheme based on compressive sensing in wireless sensor networks. Ad Hoc Netw 70:73–84
15. Liu Z, Han YL, Yang XY (2019) A compressive sensing–based adaptable secure data collection scheme for distributed wireless sensor networks. Int J Distrib Sens N 15(6):1550147719856516
16. Wang M, Xiao D, Ao Z (2019) A novel privacy-preserving data gathering scheme in wsn based on compressive sensing and embedding. In: ICC 2019 - 2019 IEEE international conference on communications (ICC), pp 1–6

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Haoran Hu is an undergraduate student in the College of Information Science and Engineering at Hunan Normal University, Changsha, China 410081. Haoran was also a visiting student at Saint Joseph's University during the academic year 2019-2020. His current research interests include wireless sensor network and computer algorithm.



Wei Chang is Assistant Professor at the Computer Science Department, Saint Joseph's University, Pennsylvania, USA. He received a Ph.D. in Computer Science from Temple University in 2016, and B.S. from Beijing University of Posts and Telecommunications in 2009. His research interests include social information-assisted system design, and its related security and privacy issues. Dr. Chang was General Chairs of IWMSN 2020, AFEC 2020, IWMSN 2019, TrustData 2019, and TrustData 2018, and Program Committee members for IEEE ICCCN 2020, ICDCS 2019, SmartWorld 2019, iSCI 2019, SmartWorld 2018, BigTrust 2018, TrustData 2017, ICCCN 2017, and ICA3PP 2017. Dr. Chang also serves as the coordinator of the Cybersecurity (CSEC) Program at the Department of Computer Science, Saint Joseph's University.